🧠 Should CORS live in NGINX or Backend?

NGINX is often the right place for CORS in production—but not always. And yes, you can (and usually should) handle preflight (OPTIONS) directly in NGINX.

👉 Flow:

```
Browser → NGINX → Backend (FastAPI / Node / Java)
```

👉 Instead of fixing CORS in every backend, you centralize it in **NGINX**

🚨 Why use NGINX for CORS

- ✅ Single place to manage CORS
- ✅ Works across microservices
- ✅ Avoid duplicate backend config
- ✅ Easier for Docker/K8s setups
- ✅ Cleaner security control

🔄 Request Flow (with NGINX fix)

1. Browser sends request
2. NGINX intercepts
3. NGINX:
    - Adds correct CORS headers ✅
    - Handles OPTIONS ✅
4. Backend stays clean (no CORS logic needed)
5. Browser accepts response ✅

✅ Use NGINX (recommended in many setups)

When it makes sense:

- You have **multiple microservices**
- You already use **NGINX as reverse proxy / API gateway**
- You want **centralized security rules**
- You want to **avoid duplicating CORS logic** in every service

👉 This is the common production pattern.

⚠️ Use Backend instead (or in addition)

When you should NOT rely only on NGINX:

- Different APIs require **different CORS policies**
- You need **fine-grained logic** (per user / tenant / route)
- You don't control the proxy (e.g., serverless / managed APIs)

👉 In those cases, backend-level CORS is safer.

🧩 Real-world approach

Best practice = **Edge (NGINX) handles generic CORS**
Backend handles **business-specific restrictions (if needed)**

🚀 How Preflight (OPTIONS) Works in NGINX

🔁 Flow

1. Browser sends:

```
OPTIONS /api/data
Origin: http://localhost:4200
Access-Control-Request-Method: POST
```

2. NGINX intercepts (does NOT forward to backend)
3. NGINX responds:

```
204 No Content
Access-Control-Allow-Origin: http://localhost:4200
Access-Control-Allow-Methods: GET, POST, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
```

4. Browser validates → sends real request

⚙️ Production-Ready NGINX Config (with preflight)

```
server {
  listen 80;

  location /api/ {

    # ✅ Handle PRE-FLIGHT (OPTIONS)
    if ($request_method = OPTIONS) {
      add_header 'Access-Control-Allow-Origin' "$http_origin" always;
      add_header 'Access-Control-Allow-Methods' 'GET, POST, PUT, DELETE, OPTIONS' always;
      add_header 'Access-Control-Allow-Headers' 'Authorization, Content-Type' always;
      add_header 'Access-Control-Allow-Credentials' 'true' always;
      add_header 'Access-Control-Max-Age' 86400 always;
      return 204;
    }

    # ✅ Normal requests → proxy to backend
    proxy_pass http://backend:8000/;
```

```
    # ✅ Attach CORS headers to actual response
    add_header 'Access-Control-Allow-Origin' "$http_origin" always;
    add_header 'Access-Control-Allow-Credentials' 'true' always;
  }
}
```

🔥 Important Details (people miss these)

✅ 1. OPTIONS must NOT hit backend

- Faster ⚡
- Reduces load

✅ 2. Use always

Without it, headers won't be added on:

- 4xx / 5xx responses ❌

⚠️ 3. Don't blindly trust $http_origin

Better:

```
set $cors_origin "";

if ($http_origin ~* (http://localhost:4200|https://yourdomain.com)) {
  set $cors_origin $http_origin;
}

add_header 'Access-Control-Allow-Origin' "$cors_origin" always;
```

❌ 4. Never do this (your original bug)

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

🧠 Mental Model

NGINX = Gatekeeper
Backend = Business logic

- NGINX: "Is this request allowed from this origin?"
- Backend: "Is this user allowed to access this data?"

⚡ Pro Architecture Trick (avoid CORS completely)

If you serve frontend + API from same domain:

```
location /api/ {
   proxy_pass http://backend:8000/;
}
```

Frontend calls:

```
/api/data
```

👉 Same origin → **No CORS, no preflight, no headache**

✅ Final Answer

- ✓ Yes, NGINX is often the **best place for CORS in production**
- ✓ Preflight is handled by intercepting OPTIONS and returning 204
- ✓ Backend can stay clean (or handle edge cases)

🧠 Pro Tips (Interview level)

✅ Option 1 (Most common)

- Handle CORS in **NGINX**

✅ Option 2

- Handle in **API Gateway (Kong, AWS API Gateway)**

❌ Avoid

- Handling CORS in **every microservice**